

Scenario 5 - Garment Factory Attendance System

This card is your team's complete capstone brief. **Part 1** describes your client. **Part 2** says what you deliver. **Part 3** covers the defense and marks. Claim one scenario per team on the LMS sign-up sheet before the Week 3 design checkpoint.

Part 1 - Your client

Client

Mekong Apparel, about 1,800 workers on two shifts. Attendance decides pay. Contact: **Ms. Leakena**, HR system owner. She checks everything twice. Paper sheets are typed into a spreadsheet on one office PC today, and it has been lost twice.

Requirements

- **R1.** Workers clock in and out fast at gate terminals.
- **R2.** Never lose a record.
- **R3.** A worker sees only their own payslip.
- **R4.** Payslip files are encrypted.
- **R5.** Email HR if the system slows during a shift change.
- **R6.** One server failing must not stop the gate.

Load

Normal	Almost nothing between shift changes
Peak	About 900 clock-ins in 20 minutes, twice a day at 07:00 and 19:00

Data

- **Records:** `workers(worker_id, full_name, line)`, `attendance(record_id, worker_id, clock_in, clock_out)`
- **Files:** payslip PDFs, key `payslips/2026-08/MA-01842.pdf`

Security rules

- **S1.** Attendance records are never lost. State your backup approach and retention.
- **S2.** Payslips are encrypted with a key the factory controls.
- **S3.** A worker cannot open another worker's payslip. You do not build a login system; explain how you would enforce it.
- **S4.** The attendance database is not reachable from the internet.

Alerts the client needs

Alert	Triggers when	Result
Shift change load	Load rises at 07:00 and 19:00	Capacity is added automatically
Gate not responding	Responses slow down during a shift change	Email to HR (R5)

You choose what to measure and the threshold. Put it in your configuration detail.

Design questions

1. What kind of database fits attendance written constantly and joined to workers for payroll?
2. Where do payslips live, and what happens to them if a server is replaced?
3. What happens at the gate at 07:00 when one server fails?
4. You know exactly when both spikes happen. What does that let your design do that other scenarios cannot?
5. How would you stop worker A opening worker B's payslip?
6. If a server's credentials were stolen, whose payslips could an attacker read, and what limits that?

Not in scope

Login, biometrics, payroll calculation, leave management, a mobile app. No real worker or pay data.

Part 2 - What your team delivers

Two sections. **Section 1** is your AWS design. **Section 2** is a live demo from your own laptop, separate from AWS.

Section 1 - AWS architecture and configuration

Design the AWS architecture for this client yourselves. Document it in enough detail that another team could build it from your documents alone.

#	Deliverable	What it contains
1A	Architecture diagram	Every AWS resource you use, how they connect, where traffic enters and where data is stored. Label it with real values: address ranges, Availability Zones, names, counts.
1B	Configuration detail	For every resource, the settings you chose and a one-line reason for each. Cover network and routing, firewall rules, identity and resource policies (written as real JSON), compute and capacity, storage, encryption and backups, the two alerts, and tags.
1C	Scenario justification	For every resource: what it does for this client, and what breaks without it. Show where each requirement R1 to R6 and each security rule S1 to S4 is met. Answer the six design questions.
1D	Infrastructure as code	Your design written as code, with test cases that prove it. Use any IaC tool.
1E	Price estimate	The monthly cost of your design: one line per resource with its size and price, a normal month and a peak month, the total, and what you would change to cut cost. Attach the estimate export or link.

Infrastructure as code levels. Use any IaC tool and any test tool. Each level needs the one before it.

Level	Submit	Proven by
1	Your network and firewall rules as code	The code passes the tool's own validation, and at least one test case proves a firewall rule
2	Your full architecture as code	A test case for each security rule S1 to S4 in Part 1, all passing
3	One command that runs validation, all tests and a security scan	The output shown, with every scan finding fixed or justified in one line

- Values in your code must match your configuration detail. Three are spot-checked.
- Git history is required. At the defense the IaC author changes one line live and re-runs the tests.

Section 2 - Cloudflare self-hosting demo

Publish a web app from a team laptop to the internet with no domain and no public IP. This runs live in class.

1. Create a free team Cloudflare account (no payment card, no domain) and install `cloudflared` on the laptop.
2. Run a web app for this scenario on the laptop: the starter kit app with `SCENARIO` set, or a one-page site. The app itself is not marked.
3. Publish it: `cloudflared tunnel --url http://localhost:8000`
4. Open the `trycloudflare.com` URL on a phone using mobile data. Show the clock-in page recording an invented worker.
5. Show the laptop has no public IP and no open inbound port (`ss -tlnp` or `netstat -an`).
6. Stop the tunnel. Show the URL no longer works.
7. Explain which direction the connection goes, and compare it with how your AWS design protects its own servers.

Quick tunnels have no uptime guarantee. Test the day before and name a backup laptop.

Part 3 - Defense and marks

Live defense

- **Trace a request** from the user's browser to the stored data and back through your design. Name the rule that allows each hop.
- **Misconfiguration hunt:** you are given a broken configuration. Find and fix it.
- **Change request:** the client asks for something new. Say what in your design changes and why.
- **Individual question:** each member answers one question on the part of the design they own.

Marks (100)

The capstone is marked out of 100 and is worth 25% of the course.

Item	Marks
Architecture diagram and scenario justification (1A, 1C)	24
Configuration detail and price estimate (1B, 1E)	20
Infrastructure as code (1D), 4 marks per level	12
Cloudflare self-hosting demo and explanation (Section 2)	16
Misconfiguration hunt and change request (team, live)	12
Individual defense answers	16
Total	100
Bonus (optional): real AWS build of your design	up to +12, total capped at 100

Bonus (optional), real AWS build. Build your design on AWS and show it running, live or in video. Marks depend on how much of the design runs and matches your configuration detail. You must provide teardown evidence. Read the notice below before you start.

Notice: if you create anything in AWS

Set a spending limit first. Clean up everything after the demo.

- Before you create any resource, set a **maximum spending limit on the payment card** linked to the account (a card limit or a virtual card from your bank), and create an **AWS Budgets alert**. AWS does not stop charging on its own. Your card limit is the only hard cap. In AWS Academy, stay inside the lab budget.
- **Delete every resource as soon as the demo ends:** instances, load balancers, databases, NAT gateways, Elastic IPs, buckets, snapshots, AMIs and stacks.
- Submit a final resource list showing nothing is left running. Charges from forgotten resources are your team's responsibility.

Rules

- **All data is invented.** No real names, records, documents or photos of people.
- **The application is not marked.** Do not build a login system.
- **Never show or commit** an AWS or Cloudflare account ID, access key, API token or password in slides, documents, code or video.

Checklist before the defense

- ☐ Architecture diagram labelled with real values
- ☐ Configuration detail for every resource, policies in JSON, a reason on every setting
- ☐ Every requirement R1 to R6 and security rule S1 to S4 traced to your design
- ☐ Both alerts designed: what is measured, the threshold, who is told
- ☐ All six design questions answered
- ☐ Price estimate attached: per resource, normal and peak month, total
- ☐ IaC validates and every test case passes, with git history
- ☐ Team Cloudflare account created, tunnel demo rehearsed, backup laptop named
- ☐ Each member knows which part of the design they own
- ☐ *(Optional, bonus)* Real AWS build: card limit set before building, running design shown, everything deleted, resource list submitted